Emergent State Machines: A Deterministic Control Framework for Interpretable Situational Reasoning

Richard Hines
Independent Researcher
Tacoma, Washington, USA
richardhinestacoma@gmail.com

Abstract

This paper presents the mathematical foundations of Emergent State Machines (ESMs), a deterministic control framework for interpretable situational reasoning in complex systems. The ESM formalism defines both individual machines and networks of interacting machines that exchange state, signals, or control across well-defined boundaries while preserving a common underlying structure independent of domain-specific logic.

The central organizing unit of this framework is the *turn*: a complete reasoning frame in which observations from the environment are transformed into signals, assembled into an interpretable state vector, projected into policy-relevant coordinates, evaluated for relevance, and, when warranted, mapped deterministically to an action or other outcome. Within each turn, raw observations are converted into direct and derived signals defined by domain experts. These signals form an explicit state vector representing the system's current situation. A projection mechanism then expresses that state in policy-relevant coordinates, enabling deterministic policy selection without collapsing reasoning into latent or opaque representations.

By structuring system behavior as sequences of turns, ESMs make reasoning processes inspectable, replayable, and auditable. Repeated turns generate trajectories through interpretable and policy-relevant state space over variable real-world time, allowing policies to guide the system toward desirable stable regions and away from undesirable ones.

Unlike architectures that entangle probabilistic interpretation with consequential action, the ESM preserves a structural boundary between state representation and authoritative policy selection. Projection interprets; relevance gating determines whether a situation warrants decision; deterministic policy authorizes action. This separation supports traceability, failure localization, and controlled system evolution through versioned revision of signals, projections, gating logic, and policies. The resulting framework provides a mathematically structured approach to decision systems whose reasoning must remain visible, reproducible, and governable across domains including clinical monitoring, learning systems, and operational control.

The canonical specification is available at `github.com/emergent-state-machine/esm-spec`.

Contribution Statement. This paper makes five principal contributions. 1. It defines the turn as the atomic unit of reasoning in a decision system, providing a formal object for inspection, replay, and audit. 2. It introduces a deterministic state-space formulation in which observations are transformed into signals, state vectors, projected representations, relevance states, and outcomes through an explicit reasoning chain. 3. It formalizes decision processes as trajectories through interpretable and policy-relevant state spaces over variable real-world time. 4. It establishes layered reasoning through explicit state initialization and handoff across interacting systems. 5. It introduces Instrumented Deterministic Evolution (IDE), a regime in which system behavior evolves only through explicit, versioned modification of architectural components.

1 Introduction

Many contemporary decision systems operate as opaque pipelines in which complex transformations produce outputs without exposing the intermediate reasoning steps that led to those outputs. In domains such as clinical care, education, manufacturing, and operational control, this opacity creates a practical problem: professionals remain responsible for consequential decisions, yet the computational systems supporting those decisions often do not provide an inspectable account of how those decisions were reached. Threshold-triggered alerting systems partially address this problem by surfacing selected measurements, but they typically expose signals rather than situations. Human operators must still reconstruct the situational meaning of those signals for themselves.

The Emergent State Machine (ESM) framework addresses this limitation by structuring automated reasoning into discrete, inspectable units called *turns*. Although this paper introduces Emergent State Machines as a class of architectures, analysis is often conducted at the level of a single machine. In practice, complex systems are composed of interacting ESMs that exchange state, signals, or control across well-defined boundaries, forming hierarchical structures in which higher-level behavior emerges from coordinated subsystems [57]. The formalism presented here therefore applies both to individual machines and to networks of interconnected ESMs.

A turn is the fundamental reasoning frame of the system. During a turn, observations are collected from the environment, transformed into direct and derived signals, assembled into an interpretable state vector, projected into policy-relevant coordinates, evaluated through relevance gating, and, when warranted, mapped deterministically to an action or other outcome. The turn therefore serves as the minimal object required to inspect, replay, reproduce, and audit system reasoning.

This framing is important for both theory and practice. Theoretically, it allows the ESM to be modeled as a dynamical system evolving through interpretable and policy-relevant state spaces, consistent with foundational work in control and decision theory [32, 13]. Practically, it gives system designers and domain experts a concrete artifact that can be logged, examined, and revised. Rather than treating reasoning as an invisible process inside a model, the ESM makes each reasoning step explicit. Each turn packages the information available to the system, the transformations applied to that information, the policy dimensions activated, the relevance conditions satisfied, and the outcome produced. A sequence of turns therefore provides both a trajectory through state space and a structured record of how the system's reasoning unfolded.

The architecture developed here differs from systems that entangle probabilistic interpretation with authoritative action selection. In the ESM, observations, signals, state representation, policy projection, relevance gating, and policy action remain structurally separated. Projection expresses what the current situation means in policy-relevant terms by evaluating the alignment of the state with defined decision patterns. Relevance gating determines whether the current situation warrants authoritative decision at all. Policy then governs which action, if any, is authorized. This separation preserves interpretability and makes system behavior traceable, aligning with calls for transparent and inspectable AI systems [21]. The resulting reasoning chain can be inspected in both directions: from observations forward to outcome, and from outcome backward to the observations and rules that produced it. Because the architecture is deterministic at the authoritative layer, system behavior is replayable under fixed versions of its governing artifacts.

Time is explicit in this framework in two distinct senses. The sequence of turns provides a discrete, inspectable reasoning order, while real-world time between turns may vary depending on when observations arrive or when conditions warrant reevaluation. This distinction allows the ESM to represent persistence, abrupt change, and temporally decaying relevance without collapsing reasoning into a continuous opaque process. In this respect, the architecture reflects core principles of feedback systems and cybernetics, in which system behavior emerges through iterative interaction between state and environment [65, 7].

This paper presents the mathematical foundations of that architecture. It first defines the turn as the atomic unit of reasoning, then formalizes the transformation from observations to signals, state vectors, projected policy coordinates, relevance-gating states, and outcomes. These components may be summarized as follows:

- observations, representing measurements or events in the monitored environment
- signals, which are direct or derived quantities computed from observations
- state vectors, organizing signals into a structured representation of the current situation
- projection into policy space, evaluating the state relative to defined decision dimensions

- relevance gating, determining whether the current situation warrants policy evaluation
- policy selection and outcome determination, governing authoritative system response
- state evolution, occurring as the system receives new observations over time
- temporal dynamics, including persistence, volatility, abrupt change, and decay of relevance

By making each stage explicit and inspectable, the ESM provides a transparent alternative to black-box decision systems. Each outcome can be traced through a deterministic chain linking actions to rules, signals, and raw observations. The architectural principles underlying the Emergent State Machine draw on ideas from control theory, dynamical systems, software architecture, and research on situation awareness and AI governance, while extending these traditions by treating signals as explicit, designable distinctions that define the structure of the system's state space. The relationship between these traditions and the ESM architecture is discussed in Section 19.

2 ESM as a Discrete-Time System

An Emergent State Machine may be understood as a discrete-time system evolving through successive updates to an explicitly constructed state. At each step, observations are transformed into structured signals, which are assembled into a state vector x_t, producing a sequence of states $\{x_t\}$ indexed by discrete steps in the system's reasoning process.

Unlike classical formulations in which state is inferred from observations, the ESM defines its state explicitly through signal construction and structured representation. State evolution therefore depends not only on incoming observations, but also on the definitions of signals and the policy structure through which those signals are interpreted.

From a control-theoretic perspective, the ESM is a dynamical system whose state transitions are governed by a structured update process. In contrast to classical system identification approaches, which infer system structure from observed behavior [40], the ESM makes both the state representation and the transformation structure explicit and inspectable.

This formulation raises a natural question: what constitutes a single step in the system's evolution? To answer this, we introduce the concept of a turn as the fundamental unit of reasoning within the ESM.

3 The Turn as the Fundamental Unit of Reasoning

The Emergent State Machine organizes reasoning into discrete computational units called *turns*. A turn represents one complete cycle of observation, interpretation, relevance evaluation, and, when warranted, policy selection. Rather than treating system reasoning as a continuous and

opaque transformation, the ESM treats each turn as an explicit object that can be inspected, replayed, and reproduced.

This discrete structuring of reasoning can be understood as a computational formalization of process-oriented accounts of system behavior, in which evolution occurs through successive transformations rather than as a single global mapping. In complex systems, outcomes often cannot be determined without stepwise evaluation of intermediate states, a property consistent with the notion of computational irreducibility [67]. The turn provides the minimal unit at which such intermediate reasoning remains explicit and inspectable.

Although the turn is defined at the level of a single Emergent State Machine, the same structure applies when multiple ESMs interact. In such systems, turns remain locally complete reasoning units, while coordination across machines occurs through the exchange of signals, state, or control at well-defined boundaries.

At time t, a turn may be represented abstractly as

$$T_t = (o_t, S_t, x_t, y_t, g_t, r_t) \tag{1}$$

where

- o_t denotes the observations available at time t,
- S_t denotes the set of direct and derived signals computed from those observations,
- x_t denotes the interpretable state vector constructed from those signals,
- y_t denotes the representation of the state in policy-relevant coordinates,
- g_t denotes the relevance-gating state,
- r_t denotes the turn outcome.

When an action is authorized, the action itself is denoted separately by a_t and is one possible component of the turn outcome.

This formulation makes explicit that a turn is not merely a timestamped input or output. It is the full reasoning frame of the system at that moment. Each turn contains the information needed to reconstruct the system's situational interpretation, the relevance conditions applied to that interpretation, and the policy logic that produced its outcome.

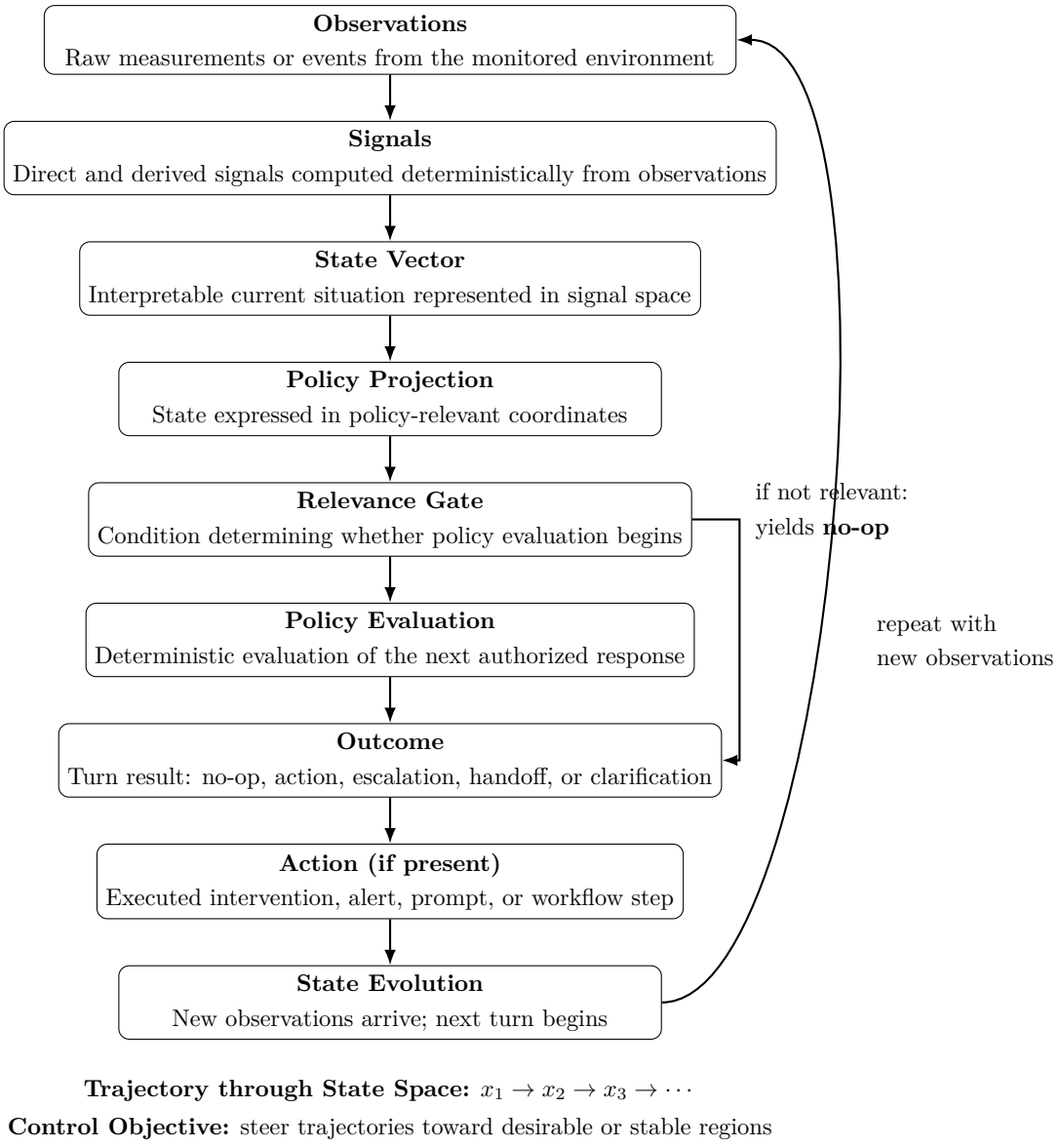

Figure 1: Core reasoning loop of an Emergent State Machine. Each turn transforms observations into signals, constructs a state vector, projects the state into policy coordinates, evaluates relevance, and, when warranted, selects an action or other outcome.

In practical deployments, the first operational turn need not begin from an empty case state. A turn may begin with initialized contextual information provided by prior system structure, a distinct intake process, or an upstream ESM. What is required is not emptiness but explicitness: the initialized conditions, the observations available, and the versioned reasoning artifacts governing the turn must all remain inspectable.

A sequence of turns forms the temporal structure of ESM reasoning:

$$T_1 \to T_2 \to T_3 \to \dots \tag{2}$$

This sequence corresponds to the evolution of the monitored system through state space. Because each turn remains explicit and inspectable, the system's reasoning process can be analyzed both at the level of individual turns and at the level of trajectories formed across many turns. In multi-machine systems, these trajectories may extend across interacting ESMs, with state, signals, or intermediate representations passed between machines. Each ESM maintains its own turn sequence, while overall system behavior emerges from the coordinated evolution of these interacting sequences.

The concept of the turn plays two roles in the architecture. First, it is the atomic object of inspection, audit, and verification. Second, it is the temporal unit through which state trajectories are generated. In this sense, turns connect the operational behavior of the system to its mathematical formulation: what evolves over time is not an abstract hidden process, but a sequence of explicit reasoning steps whose contents remain available for replay and analysis.

The turn also establishes a well-defined ordering of system behavior, allowing sequences of operations to be reasoned about explicitly. This perspective aligns with approaches in distributed systems that emphasize the importance of event ordering and traceability in understanding system execution [37]. The explicit structure of turns further supports reproducibility and correctness, as identical inputs and definitions yield identical outcomes under fixed system versions. This aligns with state-machine approaches to system design in which correctness is achieved through deterministic execution over well-defined state transitions [54].

Importantly, the structure of the turn is invariant across domains. The same sequence of observation, signal construction, projection, gating, and policy evaluation applies regardless of the subject matter being evaluated. Domain specificity is introduced exclusively through the definition of signals, the construction of derived signals, and the primitive functions used in projection.

This implies that generalization in the Emergent State Machine does not occur through parameter adaptation within a fixed model, but through reuse of a common reasoning structure across domains. Different domains instantiate the same architectural loop by redefining signals and projection functions while preserving the underlying control structure.

This implies that the unit of generalization in the Emergent State Machine is not the input

domain, but the structure of the decision process itself. This property is supported by empirical observations from early system development. An initial implementation of the ESM core loop was used to evaluate student reasoning in fraction multiplication tasks through structured signals and deterministic evaluation criteria, and was later extended to support Claim–Evidence–Reasoning (CER) writing evaluation without modification to the core reasoning loop. Only the signal definitions, derived signal constructions, and primitive functions used in projection were changed, while the underlying control structure remained unchanged.

In this sense, the Emergent State Machine is a reusable control architecture in which domains are instantiated through signal and projection design rather than through modification of the core reasoning process.

4 Observations and Signals

Each turn of an Emergent State Machine begins with observations obtained from the environment being monitored. Observations represent measurements, events, or reported information describing the system at time t.

Observations are denoted

$$o_t \tag{3}$$

where o_t represents the collection of raw inputs available to the system during turn t. These inputs may originate from sensors, instruments, logs, or structured human reporting.

Observations themselves are not directly used for policy evaluation. Instead, they are transformed into *signals*, which provide structured, policy-relevant quantities derived from the available observations.

Signals fall into two categories: *direct signals* and *derived signals*.

4.1 Direct Signals

Direct signals correspond directly to measured or reported quantities within the observations. A direct signal is defined as

$$s_i^{direct} = m_i(o_t) \tag{4}$$

where m_i is a deterministic measurement function extracting the relevant value from the observations.

Examples of direct signals include quantities such as heart rate, respiratory rate, oxygen saturation, or a recorded student response in an educational setting.

Direct signals represent the immediate measurable properties of the system at the current turn.

4.2 Derived Signals

Derived signals represent deterministic computations defined over observations or other signals. These signals capture relationships, trends, persistence conditions, rates of change, or structural properties that are not directly observable but can be computed from available measurements.

A derived signal may be written as

$$s_j^{derived} = f_j(o_{t-k}, \dots, o_t) \tag{5}$$

or equivalently

$$s_j^{derived} = f_j(s_1, s_2, \dots) \tag{6}$$

where f_j is a deterministic function defined by system designers or domain experts.

Derived signals allow the system to represent domain-specific quantities such as ratios, differences, persistence conditions, temporally windowed volatility, or rates of change. For example, a common clinical derived signal is the shock index

$$\text{shock_index} = \frac{HR}{SBP}. \tag{7}$$

Derived signals may therefore incorporate temporal structure by referencing observations across multiple turns.

4.3 Signal Set

The complete set of signals available during turn t is denoted

$$S_t = \{s_1, s_2, \dots, s_n\}. \tag{8}$$

Signals provide the structured measurement interface between raw observations and the system's internal representation of the current situation. While classical information-theoretic formulations treat signals as carriers of information independent of meaning [55], the ESM treats signals as explicitly defined distinctions that determine how the system partitions and interprets its environment [58]. Following Bateson, such distinctions become meaningful only insofar as they constitute "differences that make a difference" for interpretation and action [8].

In this sense, signals are not neutral measurements but designed distinctions that define what the system is able to perceive. The representational choices embodied in signal definitions determine which patterns can be detected, tracked, and acted upon. This perspective parallels findings in human-centered machine learning, where system performance depends critically on how data is labeled and structured, as demonstrated in the development of large-scale visual datasets [53].

Signal design therefore constitutes a separation of concerns at the level of representation: the system is structured around the distinctions it is intended to capture, and therefore around the aspects of the environment it is able to attend to and reason about [46]. The resulting collection of signals defines the representational structure from which state is constructed, reflecting a deliberate design choice about how observations are encoded for computation [42].

Because signals are explicitly defined and versioned, they remain interpretable and auditable across system revisions. At the same time, the selection and use of signals introduces potential feedback effects. As observed in studies of measurement and evaluation, the use of specific metrics to guide decision-making can alter the behavior of the system being measured, potentially degrading their validity over time [17]. By making signal definitions explicit and revisable, the ESM provides a framework in which such effects can be examined and managed rather than remaining implicit.

5 State Vector Construction

Signals derived during a turn are organized into a structured mathematical representation of the system's current condition called the state vector. Representing system behavior through explicit state vectors is standard practice in control and system identification theory [40].

The practice of representing system conditions through structured feature measurements derived from observable inputs is common in statistical learning and signal-processing systems [14, 50]. In the ESM architecture, these signals remain explicitly defined and inspectable rather than embedded within latent model parameters.

The state vector at time t is denoted

$$x_t \tag{9}$$

and constructed from the available signals as

$$x_t = \begin{bmatrix} s_1 \\ s_2 \\ \vdots \\ s_n \end{bmatrix}. \tag{10}$$

Each component of the vector corresponds to a signal computed from the observations available during the current turn.

The structure of the state vector is therefore not an intrinsic property of the monitored system, but a constructed representation determined by the selection and definition of signals. In this sense, the state vector reflects a design choice about how the system encodes and organizes information about its environment, rather than a directly observed property of the

system itself.

Unlike latent representations commonly used in many machine learning systems, the ESM state vector is explicit and interpretable. Each element corresponds to a signal whose definition, range, and computation are known. This design ensures that the system's representation of the current situation can be inspected, verified, and revised by domain experts. In this respect, the ESM aligns with traditions in symbolic and structured representations of cognition, in which system state is represented explicitly rather than implicitly encoded within learned parameters [44].

The state vector therefore functions as the system's formal description of the situation being monitored at turn t. It captures the relevant measurable and derived properties of the system in a structured mathematical form suitable for projection and policy evaluation. The resulting representation reflects a deliberate design choice about how observations are encoded for computation and interpretation [42].

The state vector may include both instantaneous signals and temporally derived signals, provided that all components remain explicitly defined and reproducible. In layered deployments, the state vector may also incorporate initialized contextual information inherited from prior turns or from upstream system components, so long as that inherited structure remains inspectable.

Because the state vector is constructed deterministically from signals, identical observations and initialized conditions will always produce identical state vectors under fixed signal definitions. This property supports replayability and traceability of system reasoning.

Across successive turns, the state vector evolves as new observations arrive and new signals are computed. The resulting sequence

$$x_1, x_2, x_3, \ldots \tag{11}$$

forms a trajectory through the system's state space, representing the evolution of the monitored situation over time.

6 Relevance Gating

Not every turn warrants authoritative decision. The Emergent State Machine therefore employs *relevance gating* to determine whether the current projected situation requires policy evaluation.

From a cognitive perspective, this problem corresponds to what has been described as relevance realization: the process by which a system determines which aspects of its current state are significant for action among many possible considerations [61]. The ESM formalizes this process as an explicit, inspectable component of the reasoning architecture.

A relevance gate evaluates conditions defined over the projected state:

$$g_t = G(y_t) \tag{12}$$

where G is a deterministic gating function and

$$g_t \in \{0, 1\}. \tag{13}$$

If $g_t = 1$, the situation is treated as decision-relevant and policy evaluation proceeds. If $g_t = 0$, the system does not invoke policy and the turn outcome is a no-op.

Relevance gating serves three primary purposes.

First, it prevents the system from treating every turn as equally consequential. The ESM evaluates every turn, but it does not authorize policy evaluation on every turn. This distinction preserves a separation between continuous observation and conditional intervention, consistent with event-triggered control formulations in which control action is invoked only when relevance conditions are satisfied [60]. In practical decision environments, this reflects the observation that expert practitioners do not evaluate all possible states continuously, but instead recognize situations that warrant attention and action [34].

Second, relevance gating restricts authoritative decision to situations that satisfy explicit structural conditions. These conditions may include threshold crossings, persistence requirements, transitions into or out of desirable regions, abrupt change, elevated volatility, or other deterministic criteria defined by system designers and domain experts. In this sense, relevance gating functions as a supervisory layer over decision activation, analogous to supervisory control of discrete event processes [51].

Third, relevance gating supports modular reasoning architectures. Different policy regimes or escalation pathways may become available under different gating conditions, but the activation of those regimes remains governed by explicit, inspectable functions of projected state.

Because relevance gates are explicitly defined, the conditions under which policy becomes active remain inspectable and auditable. When a particular turn becomes decision-relevant, the gating condition responsible for that activation can be traced directly to the signals and projected dimensions that produced it.

7 Projection into Policy Space

The state vector describes the current situation of the monitored system in terms of measurable signals. However, policy decisions are typically defined in terms of higher-level situational dimensions rather than individual signals. The Emergent State Machine therefore transforms the state vector into a representation expressed in policy-relevant coordinates.

This transformation is called *projection into policy space*.

Let P denote the policy projection operator. The projection of the state vector is

$$y_t = Px_t \tag{14}$$

where

- x_t is the state vector constructed from signals,
- P is the policy projection operator,
- y_t is the representation of the state in policy-relevant coordinates.

Projection is a required step of the architecture. Policy does not operate directly on raw observations or on unprojected state vectors. Instead, policy operates on a representation of state that has already been expressed in decision-relevant coordinates.

From a conceptual perspective, projection formalizes the role of interpretation within the reasoning process. The transformation from state to policy-relevant coordinates is not merely a change of basis, but an explicit encoding of how the system understands the current situation in terms of decision-relevant meaning. Across philosophical and cognitive traditions, interpretation has been treated as a primary component of understanding rather than a secondary transformation applied after perception. In this sense, projection can be understood as the structured expression of situational meaning, aligning with accounts of interpretation as constitutive of experience [26], historically conditioned and evolving [24], and organized through meaningful cognitive framing [16]. In practical reasoning, this corresponds to Aristotle's notion of *phronesis*, in which judgment depends on the interpretation of the particular situation rather than the application of universal rules [4]. More broadly, interpretive structure may reflect symbolic patterning in how situations are understood and categorized [29]. The ESM makes this interpretive layer explicit and inspectable rather than leaving it implicit within model structure.

Classical system identification provides a natural point of comparison for this formulation. In Ljung's treatment, system behavior is modeled through explicit state-space or input–output structures—including nonlinear state-space formulations [40, Ch. 5.3]—whose parameters are estimated from data via an iterative procedure involving data collection, model structure selection, parameter estimation, and validation [40, Ch. 1.3]. In this framework, the state is typically a latent or parameterized construct whose primary role is to support prediction, simulation, or control design, often through predictor-based estimation methods such as prediction-error minimization [40]. Even in its most general form, this approach remains centered on estimating latent system representations from data.

By contrast, the ESM constructs state deterministically from explicit signals and separates that construction from both interpretation (projection) and action (policy), preserving inspectability and governance. The Emergent State Machine adopts the state-space intuition that

system behavior can be analyzed as trajectories through a structured representation [40, Ch. 4.3], but departs from identification-as-estimation in several key respects. Rather than estimating a latent state or model parameters, the ESM constructs its state vector deterministically from explicit, versioned signals. Projection then expresses this state in policy-relevant coordinates, separating interpretation from both observation and action. This yields a representation that is not optimized for prediction accuracy alone, but for interpretability, replayability, and governance of downstream decisions.

In this sense, the role of state shifts from a latent variable supporting model estimation to an explicit situational construct supporting decision accountability. The projection operator does not collapse uncertainty into a single estimated model; instead, it organizes available signals into interpretable dimensions—potentially including confidence or ambiguity—that remain visible to relevance gating and policy. This structural separation enables the system to evolve its interpretive components through controlled revision without implicitly altering the authoritative decision mechanism.

Each component of y_t corresponds to the activation level of a policy-relevant dimension. These dimensions are defined by system designers or domain experts and represent structural properties of the situation being monitored.

While the core formalism does not require any particular structure within y_t, practical deployments often include additional interpretive components that are not directly observable in the raw signal set. In particular, many systems represent confidence or uncertainty as an explicit dimension of projected state. This allows the system to distinguish between situations that appear similar at the level of signals but differ in the reliability of their interpretation.

Such representations are optional and domain-dependent. Confidence may be computed deterministically from signal properties, derived from temporal stability, or estimated through auxiliary models, provided that its definition remains explicit and reproducible. When included, confidence becomes part of the projected state rather than an implicit property of the underlying signals, enabling relevance gating and policy to condition decisions on both the interpreted situation and the reliability of that interpretation.

For example, in a clinical monitoring system, projection dimensions might include degrees of respiratory instability, circulatory compromise, persistence of improvement, or confidence in current interpretation. In an educational system, projection dimensions might represent claim strength, evidence relevance, reasoning coherence, recent volatility, or confidence in a current assessment.

Projection therefore does not predict the future. Instead, it illuminates the current situation by expressing the measured state in coordinates relevant to relevance gating and policy evaluation. Representing system behavior through transformations of state vectors into decision-relevant coordinates is consistent with state-space approaches in control and estimation theory [32, 40].

In simple systems, projection may be close to an identity transformation. In more complex

systems, projection may involve richer deterministic transforms or reproducible model-assisted mappings. In either case, projection remains structurally distinct from policy and does not itself authorize action.

Because projection is defined as an explicit transformation of the state vector, its behavior remains inspectable and reproducible. Changes to the projection operator correspond to explicit revisions of the system's interpretive structure rather than hidden modifications to latent representations.

8 Policy Selection and Turn Outcomes

Once the system's current state has been expressed in policy-relevant coordinates and relevance gating has been evaluated, the Emergent State Machine determines a turn outcome.

If the current turn is not decision-relevant, then

$$g_t = 0 \;\Rightarrow\; r_t = \text{no-op}. \tag{15}$$

If the current turn is decision-relevant, deterministic policy is applied to the projected state:

$$a_t = \pi(y_t) \quad \text{when } g_t = 1 \tag{16}$$

where

- y_t is the policy-space representation of the current state,
- π is the policy selection function,
- a_t is the action selected by the system when action is authorized.

The resulting turn outcome is denoted by r_t. Possible outcomes include no-op, local action, escalation, handoff, or request for clarification. When an action is present, it is represented by a_t. These outcomes correspond to common patterns in distributed and enterprise systems, where processing steps result in routing, escalation, or delegation across system boundaries [27].

Policy rules encode the decision logic that governs system behavior. These rules may include threshold conditions, escalation logic, persistence requirements, cooldown constraints, routing logic, or explicit decisions to defer intervention under specified circumstances. In contrast to formulations that treat decision-making as global optimization, this structure reflects a bounded and structured approach to action selection, in which decisions are made through explicit evaluation of current conditions rather than through exhaustive search over possible futures [56].

Because policy is deterministic, identical projected states will always produce identical policy results under fixed policy definitions. This property ensures that system decisions remain reproducible and traceable.

The full reasoning chain for a turn can therefore be expressed as

$$r_t \leftarrow g_t \leftarrow y_t \leftarrow x_t \leftarrow S_t \leftarrow o_t. \tag{17}$$

When an action is present, the action itself is determined through

$$a_t \leftarrow \pi(y_t). \tag{18}$$

These chains allow any turn outcome to be traced backward through relevance gating, projection, state vector construction, signals, and observations.

In the Emergent State Machine architecture, policy represents the decision mechanism of the system, governing action selection based on explicit evaluation of the current state. While projection interprets the state of the monitored system and gating determines whether the situation warrants policy evaluation, only policy determines which action may be taken in response.

Across successive turns, policy selection induces a sequence of actions that defines a trajectory through the system's state space. The ESM therefore operates as a sequential decision process in which each action influences subsequent states and future decisions.

Within this process, the state vector serves as an explicit representation of the system's situational awareness, structuring relevant elements of the environment to support interpretation and action. This aligns with models of situation awareness that emphasize the perception of elements, their integration into a coherent representation, and the projection of future states [22].

However, unlike frameworks that treat sequential decision-making as a global optimization problem over trajectories, the ESM evaluates each step locally through explicit state construction and policy projection. Classical formulations instead seek to optimize cumulative reward over time [10].

In contrast, the ESM determines actions through interpretable alignment between the current state and policy-defined patterns. System behavior emerges from locally governed transitions rather than from optimization over extended horizons. This perspective is consistent with observations from naturalistic decision-making, in which expert practitioners act by recognizing and responding to structured patterns in the current situation rather than computing optimal policies over long-term futures [34].

9 State Evolution and Trajectories

As new observations arrive, the Emergent State Machine constructs new turns and updates its representation of the monitored situation. Each turn therefore produces a new state vector derived from the latest observations and any retained contextual structure.

Let o_{t+1} denote the observations available during the next turn. The system update can be written as

$$x_{t+1} = \Phi(x_t, o_{t+1}) \tag{19}$$

where Φ represents the deterministic transformation that incorporates new observations into the evolving state representation.

Successive state vectors form a trajectory through state space, a standard representation used in dynamical systems and control theory [7, 13]:

$$x_1, x_2, x_3, \ldots \tag{20}$$

Projection produces a corresponding trajectory through policy-relevant coordinates:

$$y_1, y_2, y_3, \ldots \tag{21}$$

This trajectory represents the temporal evolution of the monitored situation as it becomes relevant to decision.

The sequence of turns

$$T_1 \to T_2 \to T_3 \to \ldots \tag{22}$$

therefore corresponds to a trajectory through the system's interpretable and projected state spaces. This sequential structure reflects process-oriented accounts of system behavior in which meaning emerges through ordered sequences of states rather than through isolated observations [1].

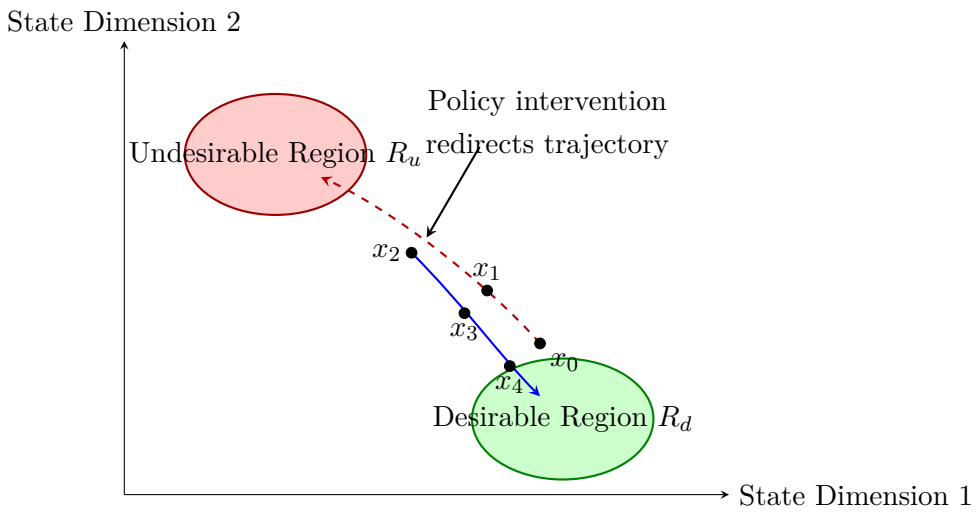

Figure 2: Example trajectory through ESM state space. Successive turns form trajectories that may converge toward desirable stable regions, persist within them, or diverge toward undesirable conditions.

Temporal relationships between observations may be incorporated into derived signals and projected dimensions. Such quantities may represent persistence, trends, rates of change, abrupt shifts, volatility over recent turns, or related measures of temporal structure. These temporal features allow the system to detect patterns that emerge over time rather than relying solely on instantaneous measurements.

Real-world time between turns may vary. Some turns may occur seconds apart, while others may be separated by minutes or longer intervals depending on the observation process and deployment context. The ESM therefore combines discrete reasoning order with variable elapsed time in the world being monitored. This distinction reflects the difference between discrete computational steps and the lived continuity of temporal processes, a distinction emphasized in philosophical treatments of time as duration rather than as a sequence of uniform intervals [12].

Because temporal relevance is not uniform, the influence of some signals or derived conditions may decay over time. A disturbance, anomaly, or diagnostic concern may be highly relevant when recent but substantially less relevant after a sustained period of stability. The architecture therefore permits explicitly defined temporally decaying influence for signals or inferred conditions whose diagnostic relevance diminishes over time.

Trajectories through projected state space may converge toward desirable regions, persist within them, oscillate near their boundaries, or diverge away from them. Policy operates on this evolving representation to guide the system toward desirable conditions and away from undesirable ones. The evolution of the system through state space is constrained not only by policy but also by the structure of possible transitions available at each step. This reflects both the concept of the adjacent possible, in which system evolution proceeds through locally

reachable configurations rather than arbitrary global transitions [33], and the role of path dependence in complex systems, where historical trajectories shape the set of future possibilities [5].

10 Layered ESMs, Initialization, and Handoff

In many deployments, Emergent State Machines operate as components within a larger system composed of interacting ESMs. These machines may operate at different levels of abstraction or over distinct aspects of the same monitored case.

A lower-level ESM may reason over a narrow local process, while a higher-level or earlier-phase ESM may construct broader contextual state. In such cases, downstream ESMs need not begin from context-free observations. Instead, they may begin from explicitly initialized state that has been produced by an upstream component or prior phase of reasoning.

This initialization may be represented abstractly as

$$x_1^{(d)} = H\Big(x_{\text{handoff}}^{(u)},\, o_1^{(d)}\Big) \tag{23}$$

where

- $x_{\text{handoff}}^{(u)}$ is the structured state provided by an upstream ESM or initializer,
- $o_1^{(d)}$ is the first local observation set of the downstream ESM,
- H is the deterministic handoff and initialization function,
- $x_1^{(d)}$ is the first downstream state vector.

This formulation clarifies two important architectural points.

First, initialization is invariant, but a distinct intake ESM is not. Every deployed ESM must begin from some initialized condition, but that initialization may be performed internally as part of the ESM's own first turn or externally through a distinct upstream process.

Second, layered ESMs interact through explicit state transfer and outcome-triggered coordination rather than through persistent shared state or implicit coupling. An upstream ESM may hand off structured state to downstream ESMs without remaining a continuing live signal source after handoff. This preserves modularity, clear provenance, and replayability while allowing higher-level systems to establish context and constraints for downstream reasoning.

In this respect, layered ESM organization aligns with hierarchical control architectures in which higher-level systems shape lower-level behavior without collapsing local autonomy [2], and with nearly decomposable system structures in which subsystems maintain relative independence while remaining coordinated within a larger whole [57].

Layered ESMs therefore allow shared situational context and specialized local reasoning to coexist without collapsing all interpretation and authority into a single undifferentiated system.

11 Desirable Regions and Stable States

Not all regions of projected state space correspond to acceptable or stable system conditions. Certain projected configurations represent preferred operational states, while others indicate instability or risk. The Emergent State Machine therefore distinguishes between desirable and undesirable regions of projected state space.

Let $\mathcal{Y}$ denote the space of all possible projected state representations. Within this space, domain experts define a subset

$$\mathcal{Y}_D \subset \mathcal{Y} \tag{24}$$

representing desirable or stable projected system conditions.

For example, in a clinical monitoring system, the desirable region may correspond not merely to isolated physiological values but to projected conditions indicating sustained respiratory stability, acceptable uncertainty, and low recent volatility. In an educational system, desirable regions may correspond to projected combinations of reasoning coherence, evidence alignment, and sufficiently stable performance over recent turns.

As the system evolves through successive turns, the sequence of projected states

$$y_1, y_2, y_3, \ldots \tag{25}$$

forms a trajectory through projected state space. This trajectory may move toward or away from the desirable region $\mathcal{Y}_D$.

Operationally, desirable region membership is often temporal rather than instantaneous. A system may enter a desirable region briefly without yet being stable in any meaningful sense. Conversely, sustained residence within a desirable region over sufficient real-world time may justify stronger claims of stability, recovery, or readiness for transition. Stability is therefore a property of trajectories over time rather than of isolated points in state space.

A useful interpretation of attractor-like behavior in the ESM is therefore temporal and policy-relative. Desirable regions may function as stable attractors when trajectories tend to enter them, remain within them, and return to them after small perturbations under the combined influence of world dynamics and policy-guided intervention, consistent with dynamical systems treatments of stability and attractors [59].

The evolution of trajectories through projected state space is further shaped by path dependence, in which prior states influence the set of reachable future configurations and the likelihood of remaining within desirable regions [5].

The ESM therefore defines a control structure in which policy does not merely respond to isolated signals but instead shapes trajectories through projected state space over time.

12 Determinism, Transparency, and Auditability

A central design principle of the Emergent State Machine architecture is that its reasoning process remains deterministic and inspectable.

The transformation chain within each turn can be expressed as

$$o_t \to S_t \to x_t \to y_t \to g_t \to r_t \tag{26}$$

with policy evaluation contributing to the outcome whenever relevance gating activates authoritative decision.

Because each stage of this transformation is explicitly defined, system outcomes can be traced through the complete reasoning pathway. This explicit ordering of transformations aligns with established principles in distributed systems, where well-defined event ordering enables reasoning about system behavior [37]. Any selected action can therefore be analyzed by examining the observations, signals, state vectors, projected dimensions, and gate conditions that produced the corresponding outcome.

This deterministic structure provides several important properties.

12.1 Replayability

Given recorded observations and fixed definitions of signals, projection, relevance gating, and policy, the system will reproduce the same turn outcome for a given turn. Interaction traces can therefore be replayed to reproduce previous system behavior. This property is consistent with state-machine approaches to system design, in which correctness is achieved through deterministic execution over well-defined state transitions [54].

Replayability enables retrospective analysis of system outcomes and allows designers to evaluate alternative projections, gating rules, or policies using historical interaction data.

12.2 Failure Localization

When undesirable outcomes occur, the source of the problem can be localized within the reasoning chain. Errors may originate from signal definitions, projection rules, relevance gates, temporal aggregation logic, or policy logic, but each stage remains independently inspectable.

Because each component of the architecture is explicit, debugging and system refinement can proceed through targeted revisions rather than opaque model retraining.

12.3 Auditability

The reasoning chain also supports formal auditing. For any turn outcome produced by the system, the full sequence of transformations from observation to projected state, relevance evaluation, and policy decision can be reconstructed.

This transparency is particularly important in domains where automated reasoning influences consequential outcomes, such as healthcare, education, or industrial operations. It aligns with broader calls for interpretable and inspectable decision systems in artificial intelligence [21].

Deterministic reasoning therefore provides a structural basis for system accountability.

13 Interpretability and Human Oversight

Traditional computational systems often produce outputs without exposing the intermediate reasoning processes that led to those outcomes. As a result, interpretability is frequently treated as a post-hoc activity in which explanations are reconstructed after decisions have been made. In contrast, the Emergent State Machine produces explicit reasoning artifacts at every turn. Rather than requiring external explanation, each outcome is accompanied by a structured representation of the observations, signals, state vector, projected interpretation, relevance gating, and policy evaluation that produced it.

This distinction shifts interpretability from an auxiliary feature to an intrinsic property of the system. Because each stage of the reasoning process is explicitly defined and preserved, the system does not merely produce decisions—it produces inspectable reasoning histories. This aligns with broader efforts in artificial intelligence to develop systems whose internal decision processes are transparent and understandable to human users [21].

The availability of explicit reasoning artifacts supports human inspection and understanding of system behavior. Users can examine not only the final outcome of a turn, but also the intermediate representations that shaped that outcome. In a clinical monitoring context, for example, a practitioner can inspect the signals derived from patient observations, the projected assessment of respiratory or circulatory stability, the relevance conditions that triggered escalation, and the policy rule that determined the recommended intervention. In an educational setting, an instructor can review the signals extracted from a student's response, the projected assessment of claim strength or reasoning coherence, and the policy conditions that led to a particular evaluation or feedback action. This structure supports forms of situation awareness in which users perceive relevant elements, integrate them into a coherent understanding, and anticipate implications for future states [22].

Interpretability in this context is not limited to passive inspection. Because the components of the reasoning process are explicit, they can also be examined in relation to one another, enabling users to construct coherent explanations of system behavior. This process aligns with accounts of sensemaking in which individuals interpret complex environments by organizing available information into meaningful structures [64]. By presenting signals, state, projections, and decisions as structured components of a reasoning chain, the ESM provides a substrate for such sensemaking rather than requiring users to infer hidden relationships within opaque models.

Crucially, interpretability within the ESM architecture supports human intervention and control. Because signals, projection functions, relevance gates, and policy rules are explicitly defined, they can be revised, extended, or constrained by system designers and domain experts. This enables a form of human oversight in which users do not merely observe system behavior but actively participate in shaping it. For example, a clinical team may refine the definition of derived signals or adjust escalation criteria in response to observed system performance, while an educational team may revise projection functions or policy thresholds to better align with instructional goals.

In this sense, the ESM supports a collaborative mode of reasoning in which human and system contributions are integrated rather than separated. The system provides structured representations and consistent evaluation, while human participants contribute domain knowledge, contextual judgment, and iterative refinement of system definitions. This perspective is consistent with views of cognition as distributed across individuals and artifacts, in which reasoning emerges from interaction between human agents and structured representations [28].

By making reasoning explicit, inspectable, and modifiable, the Emergent State Machine enables a form of interpretability that supports both understanding and action. Human users can trace how decisions were made, evaluate whether those decisions align with domain expectations, and intervene directly in the structures that govern future system behavior.

14 Human Integration

While interpretability enables human inspection and intervention, it does not by itself describe how humans and systems participate in a shared reasoning process. The Emergent State Machine supports a stronger form of interaction in which human participants and computational processes jointly contribute to the construction, interpretation, and evolution of system behavior over time.

This perspective aligns with accounts of cognition as distributed across individuals and artifacts, rather than confined to a single agent [28]. In the ESM architecture, signals, state vectors, projections, and policy rules function as structured cognitive artifacts through which reasoning is externalized and shared. The system does not merely execute decisions; it provides a persistent representational substrate within which human reasoning can be expressed, examined, and refined.

From a developmental perspective, this process reflects the role of tools in shaping cognition. Vygotsky emphasized that higher-order thinking emerges through the use of culturally mediated artifacts that structure how individuals interpret and act within their environment [63]. In the ESM, signal definitions and projection structures serve as such mediating tools, enabling domain knowledge to be encoded in forms that are both computationally actionable and human-interpretable.

Learning within this framework is inherently situated. Lave and Wenger describe learning as participation in a community of practice, in which understanding develops through engagement with shared tasks and representations [38]. The ESM provides a structured environment in which such participation can occur, as domain experts interact with system outputs, revise signal definitions, and refine policy logic in response to observed behavior.

This interaction naturally supports experiential learning cycles. Kolb characterizes learning as an iterative process involving concrete experience, reflective observation, conceptual refinement, and active experimentation [36]. In ESM deployments, historical interaction traces provide concrete experience, inspection of reasoning chains supports reflection, modification of signals and projections enables conceptual refinement, and subsequent system behavior provides a basis for further experimentation.

At the level of interpretation, this process corresponds to sensemaking, in which individuals construct meaning by organizing available information into coherent structures [64]. The ESM facilitates this process by presenting observations, signals, and decisions as components of a structured reasoning chain, allowing users to interpret system behavior in relation to domain expectations and evolving understanding.

More broadly, this integration reflects a relational view of cognition in which understanding arises through interaction between agents and their representational environment [18]. In the ESM, reasoning is not located solely within the computational system or within the human user, but emerges from their interaction through shared representations.

In practical terms, this means that system behavior evolves not only through automated processing but through ongoing human engagement. For example, a clinical team may refine derived signals or adjust escalation criteria as they observe how the system represents patient trajectories, while an instructional team may iteratively revise projection dimensions or policy rules to better align with pedagogical goals. In both cases, the system becomes a medium through which domain knowledge is progressively formalized, evaluated, and improved.

The Emergent State Machine therefore supports a mode of operation in which human expertise and computational structure are tightly integrated. Rather than replacing human judgment or requiring it to operate independently of system outputs, the architecture enables a collaborative reasoning process in which both human and system contributions are made explicit, inspectable, and subject to continuous refinement.

15 Cybernetics and Complexity

The Emergent State Machine may be situated within a broader intellectual tradition spanning cybernetics, control theory, and complexity science. These fields share a common concern with how systems regulate behavior over time through feedback, adaptation, and interaction with their environment.

Classical cybernetics established the foundational role of feedback in system behavior. Wiener characterized control systems in terms of feedback loops through which systems monitor and adjust their behavior [65]. Ashby formalized the relationship between system regulation and environmental complexity through the law of requisite variety, which states that effective control requires a system to possess sufficient internal variety to respond to external disturbances [6]. Beer extended these ideas to organizational systems, emphasizing the importance of maintaining viable structures through hierarchical coordination and feedback across multiple levels [9].

These frameworks provide a powerful account of regulation and adaptation, but they typically treat signals and system representations as given rather than as explicitly constructed and inspectable components. While cybernetic systems respond to signals, they do not generally make the definition of those signals or the structure of their interpretation available for direct examination or revision. Von Foerster emphasized the role of the observer in defining system boundaries and descriptions, highlighting that what is treated as a system depends on how it is observed and represented [62]. However, classical formulations do not provide a general architectural mechanism for making these representational choices explicit and operational within the system itself.

Complexity science extends these ideas by examining how system behavior emerges from local interactions over time. Prigogine showed that ordered structures can arise from far-from-equilibrium conditions through processes of self-organization [49]. Wolfram demonstrated that even simple rule-based systems can exhibit behavior that is computationally irreducible, requiring step-by-step simulation rather than direct prediction [67]. In biological systems, Levin has shown that signaling networks coordinate distributed processes to produce coherent large-scale behavior without centralized control [39]. These perspectives emphasize that system dynamics often cannot be fully understood through static models alone, but must be analyzed as evolving processes.

The Emergent State Machine builds on these traditions while introducing a key extension. In addition to modeling feedback, dynamics, and emergent behavior, the ESM makes the structure of perception and interpretation explicit within the system architecture. Signals are not treated as implicit inputs but as explicitly defined distinctions that determine how the system partitions its environment. Projection does not remain an internal or latent transformation but becomes an inspectable mapping from state to policy-relevant meaning. Policy, in turn, operates on these explicit representations through deterministic rules that govern action selection.

In this sense, the ESM may be understood as a modernized cybernetic architecture in which feedback, representation, and decision-making are integrated but not conflated. The system retains the regulatory structure emphasized by cybernetics and the dynamic evolution emphasized by complexity science, while extending both by making the representational and interpretive layers explicit, inspectable, and subject to controlled revision. This explicit representation layer enables a form of system design in which not only behavior, but also the

structure through which behavior is interpreted and governed, can be analyzed, tested, and evolved over time.

16 Governance

In traditional computational systems, governance is often framed in terms of outputs: whether decisions are accurate, fair, or aligned with desired outcomes. In the Emergent State Machine architecture, governance extends beyond outputs to encompass the structures that produce those outputs. Because signals, projection functions, relevance gates, and policy rules are explicitly defined, governance operates directly over the representational and decision-making components of the system.

This shift reflects the insight that measurement and representation shape system behavior. Campbell observed that when a measure becomes a target, it can distort the processes it is intended to monitor [17]. In the ESM, signal definitions determine what aspects of the environment are measured and made available for reasoning. As a result, choices about which signals to include, how they are defined, and how they are aggregated are themselves governance decisions that influence system behavior.

Similarly, classification systems are not neutral descriptions of reality but structures that organize and constrain it. Bowker and Star have shown that categories embedded in technical systems shape how phenomena are perceived and acted upon [15]. In the ESM, both signal definitions and projection dimensions function as classification mechanisms that structure how the system interprets the environment. Decisions about these structures therefore carry implications for how situations are recognized, prioritized, and acted upon.

At the level of decision-making, governance also involves the design of policy rules. Foucault emphasized that systems of knowledge and classification are intertwined with structures of power, shaping what actions are considered possible or legitimate [23]. Winner similarly argued that technological artifacts can embody political arrangements through the constraints they impose and the possibilities they enable [66]. In the ESM, policy rules encode the conditions under which actions are taken, escalated, or deferred, making them a central site of governance over system behavior.

Effective governance therefore requires not only technical correctness but also appropriate institutional structures. Ostrom's work on governing common resources emphasizes the importance of clearly defined rules, participatory oversight, and mechanisms for monitoring and revision [45]. Because the components of the ESM are explicit and modular, they can be subjected to such governance processes, allowing multiple stakeholders to participate in defining and refining system behavior over time.

The consequences of these design choices are particularly significant in domains where system outputs affect human outcomes. Benjamin has shown that technical systems can reproduce

and amplify existing inequalities when their underlying assumptions and classifications are not critically examined [11]. In an educational setting, for example, poorly designed projection dimensions or policy thresholds may systematically disadvantage certain groups of students, while in clinical contexts, signal definitions or escalation criteria may bias attention toward or away from particular patient populations.

By making the structure of perception, interpretation, and decision explicit, the Emergent State Machine enables a form of governance in which these components can be inspected, evaluated, and revised. Rather than treating system behavior as an opaque product of hidden mechanisms, the architecture exposes the elements through which behavior is produced, allowing governance to operate directly on the definitions that shape how the system understands and acts upon the world.

17 Instrumented Deterministic Evolution

While the Emergent State Machine produces deterministic outcomes within a fixed configuration, real-world systems must evolve over time as domain knowledge improves and operational requirements change. The ESM therefore supports a regime of controlled architectural revision called *Instrumented Deterministic Evolution* (IDE).

Under IDE, system behavior evolves only through explicit revision of versioned architectural artifacts. These artifacts include

- signal definitions
- derived signal computations
- projection operators
- relevance-gating functions
- policy rules
- temporal aggregation or decay logic

Any modification to these components produces a new system configuration version.

This approach reflects a broader view of learning as an iterative, instrumented process grounded in experience and evaluation. Rather than modifying behavior through implicit parameter updates, the ESM evolves through explicit revision of its representational and decision structures, allowing each change to be inspected, tested, and governed.

17.1 Structural Separation and Trainability

A central property enabling IDE is the structural separation between observation, interpretation, and authority. In the ESM, signals and projection define how the system *represents* a situation,

while policy defines how the system *acts* on that representation. This separation ensures that learning and refinement can occur within the descriptive layers without directly altering the authoritative decision mechanism.

In many contemporary systems, learned representations and decision policies are entangled within a single model, making it difficult to determine whether behavioral changes arise from improved interpretation or unintended shifts in decision logic. By contrast, the ESM isolates these concerns. As a result, learned or revised components—such as signal detectors or projection mappings—can be updated, evaluated, and versioned independently of policy.

This separation is particularly important for training. Models may be trained on recorded interaction data to improve signal extraction or projection quality, but their outputs are incorporated into system behavior only through explicit, versioned updates. Authoritative action selection remains governed by deterministic policy, preserving interpretability and control.

This structure supports forms of learning that extend beyond simple parameter adjustment. In particular, it enables revision of the system's representational assumptions themselves, consistent with accounts of double-loop learning in which underlying governing variables are reconsidered rather than merely adjusted [3].

17.2 Trajectory-Based Evaluation

Because each turn records the observations, signals, state vectors, projected states, gating results, and outcomes associated with a decision episode, historical interaction traces can be replayed under revised configurations. This enables designers to evaluate how policy revisions, projection changes, or gating changes would have affected past system behavior.

Crucially, these records form structured trajectories rather than isolated events. A sequence of turns

$$T_1 \to T_2 \to \cdots \to T_n$$

captures the evolution of system state over time, allowing designers to analyze not only individual decisions but the dynamics of behavior across entire episodes.

This enables evaluation questions such as:

- whether stability was achieved before action was authorized
- whether volatility was appropriately handled
- whether escalation or intervention occurred too early or too late
- how different configurations would have altered trajectory evolution

From this perspective, system refinement resembles a process of hypothesis testing over trajectories. Signal definitions, projection mappings, and policy rules can be treated as explicit

hypotheses about how to represent and respond to the environment, which can be evaluated against observed outcomes and revised accordingly [48].

17.3 Training and Replay Loop

Formally, let v denote a specific configuration consisting of signal definitions, projection operators, gating logic, and policy rules. The outcome produced by the system during turn t can then be written abstractly as

$$r_t^{(v)} = R^{(v)}(o_t) \tag{27}$$

where $R^{(v)}$ denotes the full deterministic reasoning chain under configuration v.

When action is authorized under that configuration, the action may be written more specifically as

$$a_t^{(v)} = \pi^{(v)}\left(y_t^{(v)}\right) \qquad \text{when } g_t^{(v)} = 1. \tag{28}$$

Evaluating the same observations under a revised configuration v' yields a potentially different outcome

$$r_t^{(v')} = R^{(v')}(o_t). \tag{29}$$

Comparing these outcomes allows system designers to study how architectural revisions affect system behavior. Because the full reasoning chain is recorded, revisions can be evaluated not only at the level of outcomes but also at the level of intermediate representations, projected dimensions, and trajectory structure.

The training and refinement process under IDE can therefore be understood as a loop:

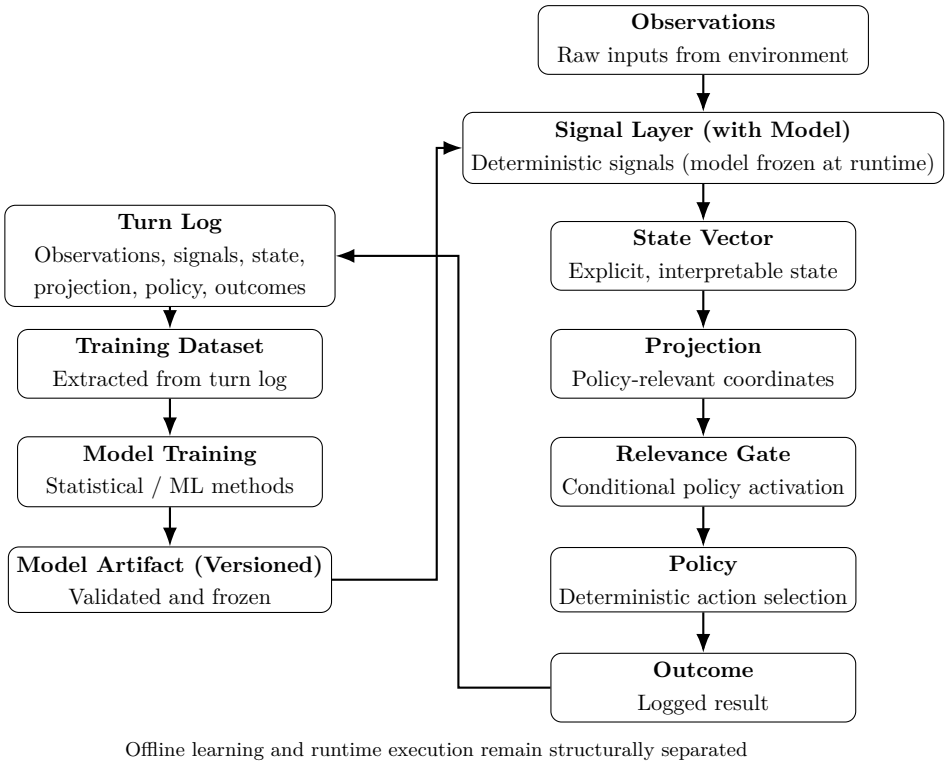

Figure 3: Training and refinement loop under Instrumented Deterministic Evolution. Interaction trajectories are recorded, analyzed, and used to update signal and projection layers, after which revised components are redeployed as versioned artifacts.

This loop reflects a cycle of experience, reflection, and revision, consistent with accounts of experiential learning in which knowledge evolves through iterative interaction with the environment [19]. At the same time, system evolution must balance refinement of existing structures with exploration of new alternatives, reflecting the tension between exploitation and exploration in adaptive systems [41].

17.4 Controlled Evolution

The key property of IDE is that behavioral change occurs only through explicit modification of versioned artifacts. System evolution therefore remains intentional and inspectable rather than emerging implicitly through hidden parameter drift.

Because learned components are introduced only through versioned updates and evaluated through replay on recorded trajectories, the system supports controlled improvement without sacrificing determinism at the decision layer.

This distinction reflects a deeper shift in how inference is treated within the system. Traditional system identification embeds inference within the system itself, often entangling representation, estimation, and decision. By contrast, the ESM externalizes inference into

instrumented, versioned components and preserves the decision state as an explicit, inspectable construct. As a result, changes to interpretation can be introduced, evaluated, and governed without implicitly altering the system's authoritative behavior.

Instrumented Deterministic Evolution therefore allows systems to improve over time while preserving replayability, traceability, and governance over consequential decision logic.

18 Boundaries of the Architecture

The Emergent State Machine framework enforces structural separation between observation, interpretation, relevance evaluation, and authoritative policy selection. While this separation provides important properties of transparency and replayability, it does not eliminate the inherent limits of modeling, measurement, or governance. The ESM should therefore be understood as an architectural framework for structured, interpretable decision processes, rather than as a universal solution to all forms of reasoning or control.

In particular, the ESM does not operate as a framework for global optimization over trajectories. Classical formulations of sequential decision-making, such as dynamic programming, seek to optimize cumulative reward across extended horizons [10]. By contrast, the ESM operates through locally evaluated decisions based on projected state and explicit policy rules. System behavior emerges from sequences of such decisions rather than from optimization over a global objective function.

18.1 Projection Quality

Projection determines how signals are interpreted as structured policy-relevant state. If signal definitions are incomplete, poorly designed, or biased, deterministic policy will faithfully act on those flawed representations.

The ESM therefore does not guarantee correctness of interpretation. Instead, it guarantees that interpretation remains explicit, inspectable, and revisable.

18.2 Policy Design

Policy encodes the authoritative decision logic of the system. Deterministic policy may still contain flawed thresholds, incorrect escalation logic, or undesirable governance assumptions.

The architecture ensures that such issues remain visible and modifiable. However, transparency alone does not guarantee that policies themselves are well designed.

18.3 Partial Observability and Clarification

Many real-world decision systems operate under partial observability, where the available signals may not uniquely determine the true system state. Formal frameworks such as partially

observable Markov decision processes (POMDPs) model this uncertainty by maintaining belief distributions over latent states [30].

The Emergent State Machine adopts a different architectural strategy. Rather than collapsing uncertainty through probabilistic belief updates within the decision mechanism, ambiguity is surfaced explicitly as a system condition. When available signals do not justify confident progress under the current reasoning constraints, the system may request clarification or additional observation before proceeding.

Clarification may arise either because relevance gating does not authorize consequential action under current conditions or because policy, once invoked, determines that additional information is required before authorizing action.

Ambiguity is therefore treated as an observable and governable system condition rather than an internal inference artifact. In contrast to structural causal approaches that focus on identifying underlying causal relationships and counterfactual dependencies [47], the ESM focuses on representing and managing the current decision context under explicit uncertainty, without requiring a complete causal model of the system.

18.4 Generative Assistance

Generative models may assist with tasks such as interpretation support, explanation generation, or interface construction. Within the ESM architecture, such models operate at the descriptive layer rather than the authoritative layer.

Generative outputs may contribute to observation processing, signal derivation, or projection support, but they cannot directly authorize actions. All consequential decisions must pass through deterministic relevance gating and policy evaluation.

This constraint allows generative capabilities to support system reasoning while preserving explicit governance over decision authority. More generally, while machine learning methods may be incorporated into signal extraction or projection layers, they do not replace the explicit policy structure that governs action selection.

18.5 Action Boundary and Deployment Constraints

The ESM architecture does not imply unrestricted automation. In safety-critical deployments, the set of allowable automated actions may be externally constrained by institutional governance, protocol boundaries, or regulatory requirements.

Accordingly, action sets may be restricted to pre-authorized, explicitly bounded operations, while higher-risk or irreversible decisions remain reserved for human or supervisory authority. The architecture supports such constraints by ensuring that all authorized actions remain explicit, versioned, replayable, and externally governable.

18.6 Applicability Scope

The Emergent State Machine is most applicable in systems where automated reasoning influences externally consequential decisions. In domains where outputs are purely generative—such as creative text production—the strict separation between interpretation and authority may be unnecessary.

The architecture therefore represents a design discipline for systems in which interpretability, accountability, and governance of automated decisions are critical requirements. It is best understood as complementing, rather than replacing, statistical learning, causal modeling, and optimization-based approaches within a broader ecosystem of decision-support methods.

19 Related Work

The Emergent State Machine (ESM) draws on ideas from control theory, dynamical systems, system identification, signal processing, software architecture, situation awareness research, and recent work on AI interpretability and governance. While each of these traditions contributes foundational concepts, the ESM combines and extends them through a distinctive architectural separation between measurement, interpretation, relevance, and authoritative policy selection.

19.1 Control and State-Space Systems

Control theory represents system behavior through state-space formulations in which system state evolves over time and control inputs influence system trajectories [32, 7]. Classical approaches distinguish between measurement, state estimation, and control, often using observers to infer latent system state and controllers to determine actions.

The ESM adopts the state-space perspective but reinterprets the role of state. Rather than estimating latent state through probabilistic inference, the ESM constructs an explicit, interpretable state vector from deterministically defined signals. Control is similarly restructured: instead of continuous control laws, the ESM employs discrete, policy-governed action selection activated through relevance gating. This preserves the trajectory-based reasoning of control theory while ensuring that both state representation and decision authority remain inspectable.

19.2 System Identification and Signal Construction

System identification frameworks model system behavior by constructing representations of state from observed data [40]. In these approaches, state is often latent and optimized for predictive performance. Signal processing and detection theory similarly focus on extracting meaningful structure from noisy observations [50].

The ESM builds on these ideas by introducing explicitly defined signals as the foundation of state construction. Rather than learning latent representations, signals are specified, versioned,

and auditable. This transforms the representation problem from one of inference to one of transparent construction, allowing domain experts to inspect and revise how observations are translated into structured state.

19.3 Representation Learning

Machine learning systems commonly transform raw observations into latent feature representations that support prediction and classification tasks [14]. These representations are typically learned through optimization and are often not directly interpretable.

The ESM replaces latent feature representations with explicitly defined signals and state vectors. Meaning is not compressed into opaque embeddings but expanded into named, interpretable components. This shift prioritizes semantic transparency over representational efficiency, enabling reasoning processes that can be inspected, audited, and revised.

19.4 Decision Theory and Uncertainty

Classical decision theory often models decision-making under uncertainty as optimization over expected outcomes, distinguishing between risk, where probabilities are known, and uncertainty, where they are not [35]. Behavioral research has shown that human decision-making frequently departs from these models, relying instead on heuristics and bounded rationality [31]. Alternative perspectives emphasize that decision strategies are adapted to environmental structure rather than globally optimal in all contexts [25].

The ESM departs from these frameworks by externalizing uncertainty into explicit system state rather than resolving it internally through probabilistic optimization. Ambiguity, conflicting signals, or insufficient evidence are represented directly within the system and may lead to deferral, clarification, or constrained action. In this sense, the architecture treats uncertainty as a condition to be governed rather than a quantity to be collapsed.

19.5 Naturalistic Decision Making

Research in naturalistic decision making has emphasized that expert decision-makers often rely on pattern recognition and situational interpretation rather than formal optimization [34]. Decisions emerge from the recognition of familiar structures within a situation and the selection of actions consistent with those structures.

The ESM aligns with this perspective by organizing reasoning around structured state representations and policy-relevant patterns rather than optimization over hypothetical futures. Projection expresses state in interpretable coordinates, and policy rules encode responses to recognizable configurations. This supports decision processes that are grounded in structured situational interpretation rather than abstract optimization.

19.6 Event-Triggered and Supervisory Control

Event-triggered control frameworks invoke control actions only when specific conditions are satisfied, rather than continuously [60]. Supervisory control theory similarly governs the activation of discrete event processes through higher-level control logic [51].

The ESM generalizes these ideas through the concept of relevance gating. While the system evaluates every turn, policy evaluation is conditionally activated only when projected state satisfies explicit structural conditions. This separates continuous observation from conditional intervention and provides an inspectable mechanism for determining when decisions become consequential.

19.7 Hierarchical and Modular Systems

Hierarchical system design has long been recognized as essential for managing complexity [57]. Subsequent work in hierarchical control [2] and modular software architecture [46, 20, 52] demonstrates how layered structures enable coordination across different levels of abstraction while preserving local autonomy.

The ESM extends these principles through layered ESM architectures in which upstream systems construct contextual state that is transferred to downstream systems. These systems interact through explicit state handoff and outcome-triggered coordination rather than continuous coupling. This preserves modularity, traceability, and replayability while allowing shared situational context across different levels of reasoning.

19.8 Dynamical Systems and Complexity

The representation of system evolution as trajectories through state space is central to dynamical systems theory, where concepts such as attractors, stability regions, and phase-space trajectories describe how systems evolve over time [59]. Complexity science further emphasizes that system evolution is often constrained by locally reachable configurations rather than arbitrary global transitions [33].

The ESM adopts this trajectory-based perspective, treating successive turns as structured evolution through interpretable and projected state space. Policy-guided interventions act to influence these trajectories while remaining constrained by the system's current configuration and available transitions.

19.9 Cybernetics and Systems Theory

Cybernetics and systems theory established foundational principles of feedback, regulation, and hierarchical organization in complex systems [65, 6, 9]. These traditions emphasize how systems maintain stability through feedback and adapt to environmental variation.

The ESM builds on these ideas while introducing an explicit representational layer. Rather than treating signals and system descriptions as implicit components of feedback loops, the architecture makes them explicit, inspectable, and subject to revision. This enables governance not only over system behavior but over the structures through which behavior is interpreted and controlled.

19.10 Situation Awareness

Situation awareness research characterizes human decision-making as a process of perceiving relevant elements, comprehending their meaning, and projecting future states [22]. This framework emphasizes the importance of structured interpretation in complex environments.

The ESM provides a computational analogue to this process. Observations are transformed into signals, signals are organized into an explicit state representation, and projection expresses that state in policy-relevant coordinates. Unlike human cognition, however, this process remains fully explicit and reproducible, enabling systematic analysis and revision of situational reasoning.

19.11 Interpretability and Governance

Recent work in AI interpretability and ethics has highlighted the risks of opaque decision systems [21, 43]. Many contemporary architectures embed both representation and decision logic within learned models, making it difficult to trace outcomes to specific inputs or assumptions.

The ESM addresses these concerns by enforcing a strict separation between interpretation and authority. Signals, state representations, projections, and policies are defined as explicit, versioned artifacts. Decisions can therefore be traced through a complete reasoning chain from observation to action, enabling auditability, failure localization, and controlled system evolution.

19.12 Distinguishing Features of the ESM

Across these traditions, the Emergent State Machine introduces a distinctive synthesis characterized by explicit architectural separation and structured reasoning:

- state is constructed through **explicitly defined, interpretable signals** rather than inferred as a latent variable
- projection expresses state in **policy-relevant coordinates** without collapsing interpretation into decision-making
- relevance gating determines when policy evaluation is authorized, **separating continuous observation from conditional intervention**
- policy selection is **deterministic, inspectable, and governed by explicit rules** rather than optimized over implicit objectives

- system evolution occurs through versioned modification of signals, projections, and policies under **Instrumented Deterministic Evolution**

Together, these elements produce an architecture in which automated reasoning remains transparent, reproducible, and governable even as systems grow in complexity and operate in uncertain domains.

20 Conclusion

This paper has presented the mathematical and architectural foundations of the Emergent State Machine (ESM), a deterministic framework for interpretable situational reasoning in complex systems. The architecture organizes reasoning into discrete units called turns, each of which transforms observations into signals, constructs an explicit state vector, projects that state into policy-relevant coordinates, evaluates relevance, and, when warranted, deterministically produces an action or other outcome.

By structuring system behavior as a sequence of turns, the ESM makes reasoning processes inspectable, replayable, and auditable. State evolution becomes visible as trajectories through interpretable and projected state space, allowing policies to guide system behavior toward desirable regions and away from unstable conditions over variable real-world time.

A central contribution of the architecture is the preservation of structural boundaries among interpretation, relevance evaluation, and authority. Projection mechanisms describe the current situation, relevance gating determines whether the situation warrants policy evaluation, and deterministic policy governs consequential action. This separation ensures that system behavior remains traceable to explicit architectural artifacts rather than embedded within opaque model parameters.

Because signals, projections, gates, temporal logic, and policies are explicitly defined and versioned, the architecture supports a regime of Instrumented Deterministic Evolution in which behavioral change occurs only through intentional revision of system components. System evolution therefore becomes controlled, inspectable, and governable rather than implicit or emergent through hidden parameter updates.

Layered deployments extend these properties by allowing initialized state to be transferred across interacting ESMs without collapsing modularity or obscuring provenance. This enables local reasoning, shared contextual state, temporal persistence, and deterministic authority to coexist within a unified and inspectable system architecture.

The Emergent State Machine does not eliminate the inherent challenges of modeling complex, uncertain domains. However, it provides a disciplined architectural framework in which automated reasoning remains transparent, reproducible, and subject to explicit governance. In systems where probabilistic interpretation influences consequential decisions, maintaining

clear structural boundaries among description, relevance, and authority is not merely a design preference, but an architectural requirement.

By making the processes of perception, interpretation, and decision explicit and inspectable, the ESM enables a new class of systems in which reasoning itself can be analyzed, evaluated, and deliberately evolved over time.

References

[1] Andrew Abbott. *Time Matters: On Theory and Method.* University of Chicago Press, 2001.

[2] James S. Albus. Theory and practice of hierarchical control. In *IEEE Conference*, 1981.

[3] Chris Argyris and Donald A. Schon. *Organizational Learning: A Theory of Action Perspective.* Addison-Wesley, 1978.

[4] Aristotle. *Nicomachean Ethics.* Oxford University Press, 2009. Translated by W. D. Ross.

[5] W. Brian Arthur. *The Nature of Technology: What It Is and How It Evolves.* Free Press, 2009.

[6] W. Ross Ashby. *An Introduction to Cybernetics.* Chapman & Hall, 1956.

[7] Karl Johan Åström and Richard M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers.* Princeton University Press, 2010.

[8] Gregory Bateson. *Steps to an Ecology of Mind.* Chandler Publishing Company, 1972.

[9] Stafford Beer. *Brain of the Firm.* Allen Lane, 1972.

[10] Richard Bellman. *Dynamic Programming.* Princeton University Press, 1957.

[11] Ruha Benjamin. *Race After Technology.* Polity, 2019.

[12] Henri Bergson. *Time and Free Will.* Macmillan, 1910.

[13] Dimitri P. Bertsekas. *Dynamic Programming and Optimal Control.* Athena Scientific, 1995.

[14] Christopher M. Bishop. *Pattern Recognition and Machine Learning.* Springer, 2006.

[15] Geoffrey C. Bowker and Susan Leigh Star. *Sorting Things Out.* MIT Press, 1999.

[16] Jerome Bruner. *Acts of Meaning.* Harvard University Press, 1990.

[17] Donald T. Campbell. *Assessing the Impact of Planned Social Change*. Dartmouth College, 1979.

[18] Anna Ciaunica. The relational self. *Phenomenology and the Cognitive Sciences*, 2020.

[19] John Dewey. *Experience and Education*. Macmillan, 1938.

[20] Edsger W. Dijkstra. On the role of scientific thought. *Selected Writings on Computing*, 1974.

[21] Finale Doshi-Velez and Been Kim. Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*, 2017.

[22] Mica R. Endsley. Toward a theory of situation awareness in dynamic systems. *Human Factors*, 37(1):32–64, 1995.

[23] Michel Foucault. *Discipline and Punish*. Pantheon, 1977.

[24] Hans-Georg Gadamer. *Truth and Method*. Continuum, 1975. Originally published 1960.

[25] Gerd Gigerenzer, Peter M. Todd, and ABC Research Group. *Simple Heuristics That Make Us Smart*. Oxford University Press, 1999.

[26] Martin Heidegger. *Being and Time*. Harper & Row, 1962. Originally published 1927.

[27] Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns*. Addison-Wesley, 2003.

[28] Edwin Hutchins. *Cognition in the Wild*. MIT Press, 1995.

[29] Carl G. Jung. *The Archetypes and the Collective Unconscious*. Princeton University Press, 1968.

[30] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998.

[31] Daniel Kahneman and Amos Tversky. Prospect theory: An analysis of decision under risk. *Econometrica*, 47(2):263–291, 1979.

[32] Rudolf E. Kalman. A new approach to linear filtering and prediction problems. *Transactions of the ASME–Journal of Basic Engineering*, 82:35–45, 1960.

[33] Stuart A. Kauffman. *Investigations*. Oxford University Press, 2000.

[34] Gary Klein. *Sources of Power: How People Make Decisions*. MIT Press, 1998.

[35] Frank H. Knight. *Risk, Uncertainty, and Profit*. Houghton Mifflin, 1921.

[36] David Kolb. *Experiential Learning.* Prentice Hall, 1984.

[37] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.

[38] Jean Lave and Etienne Wenger. *Situated Learning.* Cambridge University Press, 1991.

[39] Michael Levin. Bioelectric signaling in development. *Annual Review of Biomedical Engineering*, 2021.

[40] Lennart Ljung. *System Identification: Theory for the User.* Prentice Hall, 1999.

[41] James G. March. Exploration and exploitation in organizational learning. *Organization Science*, 2(1):71–87, 1991.

[42] David Marr. *Vision: A Computational Investigation into the Human Representation and Processing of Visual Information.* MIT Press, 1982.

[43] Brent Daniel Mittelstadt, Patrick Allo, Mariarosaria Taddeo, Sandra Wachter, and Luciano Floridi. The ethics of algorithms: Mapping the debate. *Big Data & Society*, 3(2):1–21, 2016.

[44] Allen Newell and Herbert A. Simon. Computer science as empirical inquiry: Symbols and search. *Communications of the ACM*, 19(3):113–126, 1976.

[45] Elinor Ostrom. *Governing the Commons.* Cambridge University Press, 1990.

[46] David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.

[47] Judea Pearl. *Causality.* Cambridge University Press, 2 edition, 2009.

[48] Karl R. Popper. *The Logic of Scientific Discovery.* Hutchinson, 1959.

[49] Ilya Prigogine and Isabelle Stengers. *Order Out of Chaos.* Bantam, 1984.

[50] Srinivasa Raghavan and Sanjeev Kulkarni. *Signal Detection and Estimation.* Springer, 2014.

[51] P. J. Ramadge and W. M. Wonham. Supervisory control of discrete event processes. *SIAM Journal on Control and Optimization*, 25(1):206–230, 1987.

[52] Trygve Reenskaug. Models–views–controllers. *Xerox PARC Technical Note*, 1979.

[53] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. Imagenet large scale visual recognition challenge. *International Journal of Computer Vision*, 115(3):211–252, 2015.

[54] Fred B. Schneider. Implementing fault-tolerant services using the state machine approach. *ACM Computing Surveys*, 22(4):299–319, 1990.

[55] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.

[56] Herbert A. Simon. *Models of Man*. Wiley, 1957.

[57] Herbert A. Simon. The architecture of complexity. *Proceedings of the American Philosophical Society*, 106(6):467–482, 1962.

[58] George Spencer-Brown. *Laws of Form*. Allen & Unwin, 1969.

[59] Steven H. Strogatz. *Nonlinear Dynamics and Chaos*. Westview Press, 2015.

[60] Paulo Tabuada. Event-triggered real-time scheduling of stabilizing control tasks. *IEEE Transactions on Automatic Control*, 52(9):1680–1685, 2007.

[61] John Vervaeke. Relevance realization, n.d. Lecture series and academic work on cognition and relevance realization.

[62] Heinz von Foerster. Cybernetics of cybernetics. In *Understanding Understanding*. Springer, 1974.

[63] Lev Vygotsky. *Mind in Society*. Harvard University Press, 1978.

[64] Karl Weick. *Sensemaking in Organizations*. Sage, 1995.

[65] Norbert Wiener. *Cybernetics*. MIT Press, 1948.

[66] Langdon Winner. Do artifacts have politics? *Daedalus*, 109(1):121–136, 1980.

[67] Stephen Wolfram. *A New Kind of Science*. Wolfram Media, 2002.